Stellar Cyber 6.6.0s is now live! Discover all the updates in the release notes!

Examples: Using the API to Perform an ElasticSearch Query

This section describes how to use the Stellar Cyber API to perform an ElasticSearch Query on a specified index.

ElasticSearch queries performed using the API are only available to Super Admin users with root scope with keys configured with the **Generate New Token** button in the **System | ORGANIZATION MANAGEMENT | Users** page. They are not available to users with scoped API keys configured in the **API Keys** tab. Refer to **Configuring API Authentication** for general requirements to use the API, as well as a list of the API endpoints that are only available to Super Admin users with root scope.

Make sure you optimize your ElasticSearch API queries using the guidance in **Optimizing Elasticsearch API Queries with Date-Math Index Notation**

ElasticSearch API Example

The following example uses a Python script to make an ElasticSearch DSL query. Your call must have the `application/json` header and the `/connect/api/data` path. Enter your own information for the arguments listed below:

Argument	Description
userid	User name of the admin making the call. Set to myuser@stellarcyber.ai in this example.
refresh_token	API key for the user name (edit a user on the **Admin** | **User Management** page to generate an API key). The script generates a JWT using the provided API key as a refresh token. Set to 2iRpBAyQYEfv77R2QtATlJN6Nvq6uzftBdzotSy2pjT-IvJTLw9aiHyh7Y2mo12IDSWc-FfHwUyPpmiHQnJrSH in this example.
Host	The URL or IP address of your Stellar Cyber server. Set to myserver.stellarcyber.cloud in this example.
Index	The index to be queried.

Argument	Description
	Refer to **Index Definitions & Details** for a summary of Stellar Cyber indices so you can reference them by name in your queries.
Query	What you're looking for in DSL

Understanding the Script

This script works as follows:

- The script sets the **host**, **userid**, and **refresh_token** parameters in **Step 1** in the sample.
- Because JWTs expire ten minutes after they are generated, this script includes logic that generates and uses a fresh JWT every time the script is run. The script runs the **getAccessToken** procedure to generate the new JWT (**Step 2** in the sample).
- The script uses the generated JWT to make a call to the data API in the **getData** procedure for a search of the **aella-eventsummary-*** index (**Step 3** in the sample).
- The script also prints the generated JWT to the screen. This, however, is not strictly necessary since the **getAccessToken** procedure already prints the status code for the call to the **access_token** API (**200** for success; **401** for failure).

```
#!/usr/bin/python3

                    import requests
                    import base64
                    import json
                    from urllib.parse import urlunparse
                    requests.packages.urllib3.disable_warnings()

                    # Step 1
                    # Add DP IP/hostname, userid, and refresh token from GUI here
                    HOST = "myserver.stellarcyber.cloud"
                    userid = "myuser@stellarcyber.ai"
                    refresh_token = "2iRpBAyQYEfv77R2QtATlJN6Nvq6uzftBdzotSy2pjT-
IvJTLw9aiHyh7Y2mo12IDSWc-FfHwUyPpmiHQnJrSH"

                    def getAccessToken(userid, refresh_token):
                    auth = base64.b64encode(bytes(userid + ":" + refresh_token,
"utf-8")).decode("utf-8")
                    headers = {
                    "Authorization": "Basic " + auth,
                    "Content-Type": "application/x-www-form-urlencoded",
                    }
                    url = urlunparse(("https", HOST, "/connect/api/v1/access_token",
"", "", ""))
                    res = requests.post(url, headers=headers, verify=False)
                    print(res.status_code)
                    return res.json()["access_token"]


                    def getData(token):
                    headers = {"Authorization": "Bearer " + token, 'content-type':
'application/json'}
                    url = urlunparse(("https", HOST, "/connect/api/data/aella-
eventsummary-*/_search", "", "", ""))
                    res = requests.get(url, headers=headers, verify=False)
                    print(res.status_code)
                    return res.json()

                    if __name__ == "__main__":

                    # Step 2: Use getAccessToken with supplied credentials to
generate JWT
                    jwt = getAccessToken(userid, refresh_token)
                    print("------------ jwt -------------")
                    print(jwt)
                    print("------------ jwt  end -------------")

                    # Step 3: use JWT token to call public API
                    data = getData(jwt)
```

```
                print("------------ call result of /connect/api/data -----------
--")
                print(data)
            print("------------ end api results -------------")
```

Finding the Index and ID

To find the `index` and `_id`:

1. Log in to Stellar Cyber.
2. Navigate to the event.
3. Click **More Info** for more information on the event. The Event Details screen appears.
4. Click the **Details** entry in the left navigation panel (or **JSON**) and scroll to the bottom of the display.
5. The `_index` field is the index, and `_id` field is the ID.

Optimizing Elasticsearch API Queries with Date-Math Index Notation

When querying Elasticsearch through the API, the scope of the index pattern in your request directly affects cluster performance. An unscoped query forces Elasticsearch to search every matching index regardless of age, which can cause significant latency and resource consumption in high-volume deployments where new indices are created daily.

This section explains why index scoping matters and how to use date-math notation to limit queries to only the indices that are relevant to your search.

The Problem with Unscoped Queries

A broad wildcard pattern such as the following instructs Elasticsearch to search *every* index matching that prefix, regardless of how old it is:

```
/connect/api/data/aella-wineventlog-*/_search
```

In active deployments, this can mean dozens or hundreds of indices spanning weeks or months of data. Most of those indices are irrelevant to a typical query, but the cluster must still open and scan each one.

> ❗ Unscoped wildcard queries can cause severe performance degradation on large clusters. Avoid using open wildcard patterns in production API integrations.

The Recommended Approach: Date-Math Index Notation

Elasticsearch supports date-math expressions in index names, which are resolved dynamically at query time. The recommended pattern limits the query to today's index and yesterday's index only:

```
/connect/api/data/<aella-wineventlog-{now/d}-*>,<aella-wineventlog-{now-1d/d}-
*>/_search
```

At query time, Elasticsearch resolves this expression to the two specific daily indices, for example:

```
/connect/api/data/aella-wineventlog-2026-05-13-*,aella-wineventlog-2026-05-12-
*/_search
```

Syntax Reference

Component	Description
`<...>`	Denotes a date-math expression. Elasticsearch resolves the expression at query time.
`{now/d}`	The current date, rounded down to the start of the day (for example, `2026-05-13`).
`{now-1d/d}`	Yesterday's date, rounded down to the start of the day (for example, `2026-05-12`).
`aella-wineventlog-...-*`	Matches all shards or partitions of that day's index.
`,`	Comma-separates multiple index targets within a single query.

Why This Improves Performance

Scoping queries to specific daily indices reduces the work the cluster must perform at every layer of query execution:

- **Fewer shards scanned.** Elasticsearch opens and queries only the shards belonging to the targeted indices.
- **Lower coordination overhead.** The coordinating node aggregates results from far fewer shards, reducing CPU and memory usage.
- **Reduced cache pressure.** Field data and filter caches are warmed only for the targeted indices.
- **Improved query latency.** Particularly important for real-time or near-real-time security event queries where response time is critical.

Extending the Lookback Window

If your use case requires a longer lookback period, extend the pattern explicitly by adding additional date-math terms rather than reverting to an open wildcard:

```
/connect/api/data/<aella-wineventlog-{now/d}-*>,<aella-wineventlog-{now-1d/d}-*>,<aella-wineventlog-{now-2d/d}-*>/_search
```



Each additional day added to the pattern increases the number of shards Elasticsearch must query. Extend the window only as far as your use case requires.

Summary

Always scope API queries to the minimum date range necessary for your search. The performance impact of an open wildcard pattern scales with the age and total volume of your data, making it unsuitable for production use in long-running deployments.

Pattern Type	Recommendation
Open wildcard (`aella-wineventlog-*`)	Avoid in production. Performance impact grows with data volume.
Date-math scoped pattern	Recommended. Limits queries to only the indices relevant to the search window.



© 2026 Stellar Cyber . All rights reserved.